Genius

Kit do QA · UAT v1 · Documento 05

Automação de testes de front

Tutorial do zero para rodar e escrever testes com Playwright contra o UAT do ERP Genius. Para quem nunca automatizou.

Versão	22/10/2026
Público	QA júnior, sem experiência com automação
Base	Pasta `e2e/` e `playwright.config.ts` da branch `qa/auditoria-e-onboarding` (commit `1a4c2934`)
Ferramentas	Playwright 1.64.1 · Node 22 (arquivo `.nvmrc`) · Chromium
Leia antes	Documentos 01 a 04 do kit (onboarding, usuários e papéis, roteiro manual, fluxo de bugs)

Os testes rodam na sua máquina, contra o UAT. Você só edita a pasta `e2e/`. Nunca aponte para produção e nunca escaneie o QR do WhatsApp.

Sumário

1	O que é um teste de front automatizado	Analogia, Playwright, headless, spec
2	O que já está pronto no projeto	Pasta e2e, os 50 testes, test.fixme
3	Preparar a máquina	Node 22, Git, VS Code, npm install, Chromium
4	Configurar o acesso ao UAT	.env.e2e, URL, senhas por papel
5	Rodar pela primeira vez	Terminal, modo UI, headed, filtro por ID
6	Ler o resultado	Relatório HTML, trace, vídeo, issue
7	Anatomia de um teste	ACE-07 linha a linha e o fixture papel
8	Escrever o seu primeiro teste	Codegen, limpeza do código, rodar
9	Boas práticas neste projeto	Nomes, seletores, dados, proibições
10	O que automatizar primeiro	P1 de ACE, PER, CFG e CRM; bloqueadores
11	Quando quebrar	Erros comuns e o que fazer
12	Fluxo de entrega	Branch, commit em e2e/, PR, CI
13	Glossário e checklists	Termos e listas de conferência

Como ler este guia

Cada passo traz o comando exato num bloco cinza. Logo abaixo vem uma caixa verde com o que você deve ver na tela. Se a tela mostrar outra coisa, pare e vá à seção 11.

```
npm run test:e2e
```

O que você deve ver
A descrição do resultado esperado fica aqui.

Nos blocos de comando, troque o que está entre `<` e `>` pelo seu valor. Exemplo: `<url-do-uat>` vira a URL que o gestor passou. As caixas coloridas seguem o resto do kit:

Azul explicação ou dica · Amarelo atenção, dá para errar aqui · Vermelho proibido, não faça · Verde exercício.

Onde está a versão curta
O guia oficial e resumido é o `e2e/README.md`. Este documento explica o mesmo conteúdo com calma, passo a passo. Se os dois divergirem, vale o README e avise o gestor.

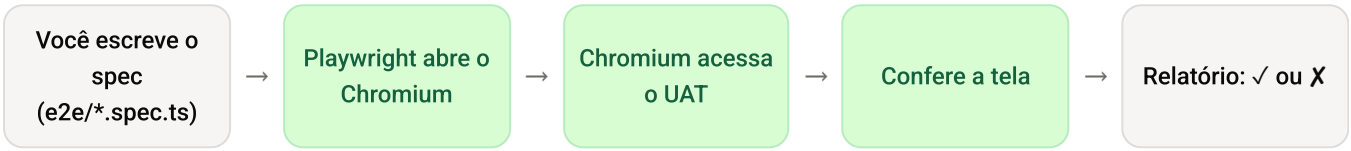
1 O que é um teste de front automatizado

É um roteiro de teste que o computador executa sozinho, num navegador de verdade, clicando e digitando como um usuário. No fim, ele diz o que passou e o que falhou.

Uma analogia

Pense numa receita de bolo. No teste manual, você lê a receita e faz o bolo. No teste automatizado, você escreve a receita num formato que um robô de cozinha entende. O robô faz o bolo do mesmo jeito todas as vezes. Quando algo dá errado, ele para e avisa: "no passo 6 a massa não cresceu".

No ERP Genius, a "receita" é algo assim: abrir `/login`, digitar o e-mail, digitar a senha, clicar em **Entrar** e conferir que o menu lateral tem os 9 itens do Vendedor, na ordem certa. Você já faz isso à mão no roteiro (documento 03). A automação repete esse roteiro em segundos, para os 14 usuários de teste.



Quatro palavras que você vai usar todo dia

▷ Playwright

Ferramenta gratuita da Microsoft que controla o navegador por código. O projeto usa a versão 1.64.1. Os testes são escritos em TypeScript, uma forma de JavaScript com tipos.

Navegador headless

"Sem cabeça": o Chromium roda sem janela. Ele carrega, clica e digita, mas não desenha nada na sua tela. É mais rápido. Para ver o navegador, use o modo UI ou `--headed`.

Spec

Arquivo que termina em `.spec.ts` e guarda os testes de um módulo. Exemplo: `e2e/login.spec.ts` tem os testes de login. Cada teste dentro dele é um cenário.

Asserção (expect)

A conferência do teste: "espero que o título Acesso negado esteja visível". O Playwright espera até 10 segundos. Se não aparecer, o teste falha.

Manual e automatizado: cada um no seu papel

O manual é bom para fluxo novo, usabilidade, texto confuso e layout. O automatizado é bom para o que se repete igual: login, menu por papel, permissões e regressão a cada versão. Ele roda em segundos, mas não pega o que ninguém mandou conferir.

Onde isso entra na estratégia de testes

O documento 14 descreve uma pirâmide. Na base ficam os testes unitários e, no meio, os de integração com banco descartável (Vitest, do time de dev). No topo ficam os testes ponta a ponta (E2E), com Playwright. Quando o documento 14 foi escrito, o E2E era "futuro". Hoje ele existe na pasta `e2e/`, e essa pasta é sua.

> ⚠ **A automação não substitui o roteiro manual**
> O primeiro ciclo continua manual. Automatize o que já passou no manual, se repete a cada versão e tem prioridade P1. Um teste automático só confere o que você mandou conferir.

2 O que já está pronto no projeto

O projeto já tem a configuração, o login automático por papel e 50 testes. Você não começa do zero: começa ampliando o que existe.

Os arquivos

Arquivo	O que faz	Você edita?
`playwright.config.ts`	Lê o `.env.e2e`, exige `E2E_BASE_URL`, define os tempos (teste 60 s, expect 10 s, ação 15 s, navegação 30 s), guarda print, vídeo e trace só de teste que falhou, gera o relatório HTML. Define dois projetos: `setup` e `chromium`.	Não. Combine antes.
`e2e/auth.setup.ts`	Roda antes de tudo. Para cada papel com senha, faz login pela tela `/login` e salva os cookies em `e2e/.auth/<PAPEL>.json`. Papel sem senha é pulado.	Raramente.
`e2e/fixtures/usuarios.ts`	Lista os 14 papéis (`PAPEIS`), os e-mails padrão do seed e lê as senhas do ambiente. Se faltar senha, explica qual variável preencher.	Só para incluir papel novo.
`e2e/fixtures/auth.ts`	Função `login(page, papel)`, função `menuPrincipal(page)` e o `test` com a opção `papel`, que começa o teste já logado.	Raramente.
`e2e/*.spec.ts`	Os testes. Um arquivo por módulo do roteiro.	Sim. É o seu trabalho.

Os 50 testes que já existem

Para ver a lista completa sem executar nada, rode o comando abaixo (ele só precisa da URL, não das senhas):

```
npx playwright test --list
```

O que você deve ver

Uma linha por teste, no formato `[projeto] › arquivo:linha › grupo › nome`, e no fim:

```
Total: 50 tests in 3 files
```

Grupo	Qtde	O que confere	Estado
`autenticar <PAPEL>` (setup)	14	Login de cada papel e gravação da sessão	Roda sempre antes
ACE-03	14	Login com sucesso e itens do menu lateral, na ordem exata, um teste por papel	Ativo

Grupo	Qtde	O que confere	Estado
ACE-07	1	Senha errada e e-mail inexistente mostram "E-mail ou senha incorretos." e ficam em `/login` (a 1ª tentativa do cenário de bloqueio)	Ativo
ACE-06	4	`/alunos`, `/financeiro`, `/configuracao/usuarios` e `/funil` sem sessão vão para `/login?callbackUrl=…`	Ativo
PER-02a–c, PER-07a–c, PER-09a–b, PER-11a–b, PER-13a–b	12	Papel digita uma URL proibida e vê "Acesso negado"	Ativo
PER-02d, PER-04a, PER-05a, PER-07d, PER-11c	5	Bugs conhecidos X2 e X3 e decisão pendente R13	test.fixme

As duas formas de "Acesso negado"

Redirecionamento

A página manda para `/acesso-negado`, que mostra o título "Acesso negado" e o link "Voltar ao início". Exemplos: `/funil` e `/diario`. No spec: `mecanismo: "redirect"`.

Na própria página

A URL não muda e a tela mostra "Acesso negado" no lugar do conteúdo. Exemplos: `/financeiro`, `/alunos` e `/empresas`. No spec: `mecanismo: "na-pagina"`.

O que está em test.fixme e por quê

`test.fixme(...)` marca um teste que o Playwright lista, mas não executa. No resultado ele aparece como pulado. O teste descreve o comportamento **esperado**, que hoje não acontece por causa de um bug conhecido ou de uma decisão que o PO ainda não tomou.

ID	Papel e URL	Motivo
PER-05a	GERENTE_COMERCIAL em `/empresas`	Bug X2: "Empresas" está no menu, mas a página nega o acesso
PER-07d	SECRETARIA em `/empresas`	Bug X2, o mesmo caso para a Secretaria
PER-02d	VENDEDOR em `/financeiro/permuta`	Bug X3: tela de erro genérica em vez de "Acesso negado"
PER-11c	GERENTE_PEDAGOGICO em `/configuracao/contratos`	Bug X3, mesma tela de erro
PER-04a	GERENTE_COMERCIAL em `/configuracao/crm`	Decisão R13: sem item no menu, mas a página abre pela URL

Quando a correção for publicada no UAT (ou o PO decidir), troque `test.fixme` por `test` e rode. Se o teste passar, ele vira proteção contra o bug voltar. Se a decisão for manter o comportamento atual, ajuste ou apague o caso.

Os nomes já usam os IDs do roteiro

Cada teste começa pelo ID do cenário em `docs/qa/03-roteiro-testes-manuais.md`. Quando um cenário vira vários testes, a letra final identifica a URL: `PER-02a` é `/alunos`, `PER-02b` é `/configuracao/usuarios`. O filtro `-g`, a planilha e a issue usam o mesmo ID.

3 Preparar a máquina

Você vai instalar quatro coisas: Node, Git, VS Code e o navegador do Playwright. Faça uma vez por máquina. Reserve cerca de uma hora.

> **O que é o terminal**
> É a janela onde você digita comandos. No Windows, use o **PowerShell** ou o **Git Bash** (vem com o Git). No Mac, use o app **Terminal**. Dentro do VS Code, o terminal abre com `Ctrl` + `` ` `` (Mac: `Control` + `` ` ``). Digite o comando e aperte `Enter`.

1 Instale o Node.js na versão 22

O arquivo `.nvmrc` do projeto diz `22`. É a versão usada pela CI. O mínimo técnico é 20.12, porque o `playwright.config.ts` usa `process.loadEnvFile`.

Windows: no site nodejs.org, página de download, escolha a versão 22 e o instalador `.msi`. Avance com as opções padrão.

Mac: no mesmo site, baixe o instalador `.pkg` da versão 22. Se você já usa o nvm, rode `nvm install` dentro da pasta do projeto: ele lê o `.nvmrc`.

Feche e abra o terminal. Depois confira:

```
node -v
```

> **O que você deve ver**
> `v22.` seguido de dois números, por exemplo `v22.12.0`. Se aparecer "não é reconhecido" ou "command not found", a instalação não terminou: reinstale e abra um terminal novo.

2 Instale o Git

Windows: baixe em git-scm.com e instale com as opções padrão. Ele traz o Git Bash.

Mac: rode o comando abaixo. Se o Git não existir, o macOS oferece instalar as "ferramentas de linha de comando". Aceite.

```
git --version
```

> **O que você deve ver**
> `git version 2.` seguido da versão, por exemplo `git version 2.47.1`.

3 Instale o VS Code e a extensão do Playwright

Baixe o VS Code em code.visualstudio.com. Abra a aba Extensões (`Ctrl` + `Shift` + `X`, no Mac `Cmd` + `Shift` + `X`), procure **Playwright Test for VSCode**, publicada pela Microsoft, e clique em Install.

> **O que você deve ver**
> Um ícone de tubo de ensaio (Testing) na barra da esquerda. Depois do passo 6, ele lista os testes da pasta `e2e/` com um botão ▶ ao lado de cada um.

4 Clone o repositório

Você precisa do convite do GitHub (documento 01). No terminal, entre na pasta onde quer guardar o projeto e rode:

```
git clone https://github.com/diogenesmendes01/erp-genius.git
cd erp-genius
```

O que você deve ver

`Cloning into 'erp-genius'...` e, no fim, o prompt de volta. Depois do `cd`, você está dentro da pasta `erp-genius`. Se o GitHub pedir login, entre com a sua conta convidada.

---

5 Instale as dependências do projeto

Este comando baixa as bibliotecas do projeto, inclusive o `@playwright/test`. Na regra do time, os agentes de desenvolvimento não rodam install: quem prepara o ambiente deles é o integrador. A sua máquina de QA é outra situação. Aqui você instala.

```
npm install
```

O que você deve ver

Alguns minutos de espera e, no fim, uma linha como `added 1234 packages in 3m` (o número varia). Linhas `npm warn` são avisos e não impedem nada. Uma linha `npm error` impede: veja a seção 11.

---

6 Baixe o navegador usado nos testes

```
npx playwright install chromium
```

O que você deve ver

Barras de progresso de download e uma linha dizendo que o Chromium foi baixado para uma pasta `ms-playwright`. Para conferir a versão do Playwright:

```
npx playwright --version
Version 1.64.1
```

⚠ **Windows: "a execução de scripts foi desabilitada"**

Se o PowerShell recusar o `npx` ou o `npm` com essa mensagem, troque por `npx.cmd` e `npm.cmd` (exemplo: `npx.cmd playwright install chromium`), ou use o terminal Git Bash. Não altere a política de segurança do Windows por conta própria: fale com o gestor.

4 Configurar o acesso ao UAT

Os testes precisam de duas informações: a URL do UAT e as senhas dos usuários de teste. Elas ficam num arquivo só seu, o `.env.e2e`, que nunca vai para o GitHub.

1 Copie o modelo

Na pasta do projeto, rode o comando do seu terminal:

```
# Git Bash ou Mac

cp .env.e2e.example .env.e2e
# PowerShell

Copy-Item .env.e2e.example .env.e2e
```

O que você deve ver

Nenhuma mensagem. No VS Code, o arquivo `.env.e2e` aparece na raiz do projeto, ao lado do `playwright.config.ts`, com o nome em cinza (cinza quer dizer "ignorado pelo Git").

2 Preencha a URL do UAT

Abra o `.env.e2e` no VS Code. Troque o exemplo pela URL que o gestor passou por canal privado. Ela começa com `https://` e não tem barra no final.

```
E2E_BASE_URL=<url-do-uat>
```

O que você deve ver

A URL abre no seu navegador comum e mostra a tela de login do ERP Genius. Confira também `<url-do-uat>/api/health`: deve mostrar `ok: true` e um `sha`.

3 Preencha as senhas

Para cada papel, preencha a linha `E2E_PASSWORD_<PAPEL>=` com a senha recebida. Sem espaço antes ou depois do `=`. Se a senha tiver espaço ou `#`, coloque-a entre aspas simples. As linhas `E2E_EMAIL_<PAPEL>` já vêm preenchidas com o padrão do seed: só mude se o gestor pedir.

```
E2E_PASSWORD_VENDEDOR=<senha-do-vendedor>
E2E_PASSWORD_PROFESSOR=<senha-da-professora>
```

O que você deve ver

O arquivo salvo, com as 14 linhas de senha preenchidas. Pode começar com só algumas: os papéis sem senha são pulados no setup (seção 5).

4 Confira que o arquivo está protegido

```
git check-ignore -v .env.e2e
```

O que você deve ver

```
.gitignore:47:.env.e2e  .env.e2e
```

Isso quer dizer que a linha 47 do `.gitignore` protege o arquivo. Se não aparecer nada, pare e avise o gestor antes de qualquer commit. Rode também `git status`: o `.env.e2e` não pode aparecer na lista.

As variáveis por papel

Cada papel usa duas variáveis: `E2E_EMAIL_<PAPEL>` e `E2E_PASSWORD_<PAPEL>`. Exemplo: `E2E_PASSWORD_GERENTE_PEDAGOGICO2`. O apelido é o mesmo do documento 02 e do roteiro.

<PAPEL>	Apelido	Papel no sistema	E-mail padrão
`ADMIN`	ADM	Administrador	admin@genius.com
`ADMIN2`	ADM-2	Segundo administrador (decisões independentes)	admin2@genius.com
`GERENTE_COMERCIAL`	GC	Gerente Comercial	gerente@genius.com
`VENDEDOR`	VEND-A	Vendedor	joao@genius.com
`VENDEDOR2`	VEND-B	Vendedor de outra carteira	vendedor2@genius.com
`VENDEDOR3`	VEND-0	Vendedor sem limite de desconto	vendedor3@genius.com
`GERENTE_PEDAGOGICO`	GP	Gerente Pedagógico	gp@genius.com
`GERENTE_PEDAGOGICO2`	GP-2	Segundo Gerente Pedagógico	gp2@genius.com
`PROFESSOR`	PROF	Professor	carla@genius.com
`PROFESSOR2`	PROF-2	Professor de outra turma	professor2@genius.com
`FINANCEIRO`	FIN	Financeiro	financeiro@genius.com
`FINANCEIRO2`	FIN-2	Segundo Financeiro (aprova acertos)	financeiro2@genius.com
`SECRETARIA`	SEC	Secretaria Acadêmica	secretaria@genius.com
`VENDEDOR_PROFESSOR`	VEND+PROF	Vendedor e Professor no mesmo usuário	venprof@genius.com

Usuários do roteiro que ainda não têm variável

GC+V (mariana@genius.com) e FIN+SEC (ana@genius.com) existem no roteiro, mas não na lista `PAPEIS`. Cenários com eles (ACE-05, ACE-10) ainda não dão para automatizar. Incluir um papel novo mexe em `fixtures/usuarios.ts`, no `.env.e2e.example` e no README: combine com o gestor.

Proibido

Commitar o `.env.e2e` ou a pasta `e2e/.auth/`. Colocar senha ou URL do UAT direto no código. Colar senha em issue, print ou chat. Apontar `E2E_BASE_URL` para produção: os testes logam, clicam e, nos próximos specs, criam dados.

Se preferir não usar arquivo, você pode definir as variáveis no terminal. O que está no terminal tem prioridade sobre o arquivo:

```
# Git Bash ou Mac

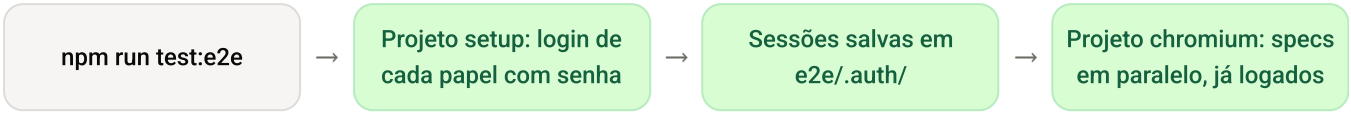
export E2E_BASE_URL=<url-do-uat>
export E2E_PASSWORD_PROFESSOR='<senha>'
# PowerShell

$env:E2E_BASE_URL = '<url-do-uat>'
$env:E2E_PASSWORD_PROFESSOR = '<senha>'
```

5 Rodar pela primeira vez

Agora você roda os 50 testes, aprende a ver o navegador trabalhando e a rodar só o que interessa.

O que acontece quando você roda



O setup faz o login uma vez por papel, pela tela `/login`, e grava os cookies em `e2e/.auth/<PAPEL>.json`. Os testes de permissão reaproveitam esses arquivos e começam já logados. Isso economiza tempo e evita gastar tentativas de login. O setup roda sempre, mesmo quando você filtra os testes.

Rode tudo, sem janela

```
npm run test:e2e
```

O que você deve ver

As linhas aparecem enquanto os testes rodam. Exemplo resumido (tempos e número de workers variam):

```
Running 50 tests using 4 workers

  ✓  1 [setup] › auth.setup.ts:23:8 › autenticar ADMIN (4.1s)
  ✓  2 [setup] › auth.setup.ts:23:8 › autenticar ADMIN2 (3.8s)
  …
  ✓ 29 [chromium] › login.spec.ts:140:7 › Login › ACE-07 senha errada … (2.6s)
  - 46 [chromium] › permissoes.spec.ts:199:10 › … › PER-05a item Empresas …
  …
  5 skipped
  45 passed (1.6m)

To open last HTML report run:

  npx playwright show-report
```

Com todas as senhas preenchidas, o esperado é 45 passaram e 5 pulados (os `test.fixme`).

Símbolo	Significado
✓	Passou.
✗	Falhou. O erro aparece logo abaixo da lista, com o arquivo e a linha.
-	Pulado: `test.fixme`, ou setup de um papel sem senha.
`did not run`	Nem começou. Acontece quando o setup de algum papel falha: o projeto `chromium` inteiro fica parado.

Quando falta uma senha

O setup daquele papel aparece com `-` (pulado) e a razão: `E2E_PASSWORD_ADMIN2 não definida: papel pulado no setup`. Os outros papéis rodam normalmente. Os testes que usam o papel sem senha falham dizendo o que preencher:

```
[e2e] Senha do papel ADMIN2 não definida. Preencha E2E_PASSWORD_ADMIN2 no arquivo .env.e2e …
[e2e] Sessão de ADMIN2 não encontrada em …/e2e/.auth/ADMIN2.json. …
```

A primeira aparece no ACE-03 daquele papel. A segunda, nos testes que começam logados com ele. Já uma senha **preenchida mas errada** faz o setup falhar, e aí nenhum spec roda. Corrija a senha.

Outras formas de rodar

O `--` separa as opções do npm das opções do Playwright. Tudo o que vem depois dele vai para o Playwright.

Comando	O que você deve ver
`npm run test:e2e:ui`	A janela do modo UI: à esquerda, os arquivos e testes, com filtro; clique no ▶ de um teste. No alto, miniaturas da tela; no centro, a tela em cada passo; embaixo, log, console e rede. Se um teste com papel falhar com "Sessão de … não encontrada", rode antes `npm run test:e2e` uma vez pelo terminal.
`npm run test:e2e -- --headed`	Janelas do Chromium abrindo e fechando rápido, uma por worker. Junte com um filtro para acompanhar com calma.
`npm run test:e2e -- -g "ACE-03"`	As 14 linhas do setup e os 14 ACE-03, um por papel. Nada de ACE-06, ACE-07 ou PER. O filtro é por texto: `-g "PER-"` pega todos os PER; `-g "PER-02a"`, só um.
`npm run test:e2e -- e2e/permissoes.spec.ts`	O setup e os 17 testes de permissão (12 ativos e 5 pulados).
`npm run test:e2e -- -g "PER-02a" --debug`	O navegador e o Playwright Inspector. O teste para antes de cada ação; o botão Step over executa uma linha por vez.

> **Cuidado com o ACE-07 e o bloqueio de login**
> O ACE-07 erra a senha do VEND-B (vendedor2) de propósito, uma vez por execução. Depois de 5 falhas em 15 minutos, o sistema bloqueia aquele e-mail, inclusive com a senha certa. Aí o setup do VENDEDOR2 falha e **nenhum spec roda**. Não rode a suíte inteira mais de 4 vezes em 15 minutos. Para repetir muito, filtre: `-g "PER"` ou o arquivo do seu spec.

6 Ler o resultado

O terminal diz o que falhou. O relatório HTML e o trace dizem por quê. Print, vídeo e trace só são guardados para testes que falharam.

Relatório HTML

```
npx playwright show-report
```

O que você deve ver

O terminal mostra `Serving HTML report at http://localhost:9323` e o navegador abre a lista de testes. No alto, filtros: Passed, Failed, Flaky, Skipped. Clique num teste para ver os passos, o erro, o print e o vídeo. Para fechar, volte ao terminal e aperte `Ctrl` + `C`.

Trace viewer, passo a passo

O trace é uma gravação completa do teste. Dá para "voltar no tempo" e ver a tela, o console e as requisições de cada momento. É a melhor evidência para o dev.

1. **Abra o teste que falhou no relatório HTML**
   Filtre por Failed e clique no nome do teste.

2. **Clique na miniatura da seção Traces**
   Fica no fim da página do teste. O Trace Viewer abre.

3. **Leia a lista de ações, à esquerda**
   Cada linha é um comando: `goto`, `fill`, `click`, `expect`. A ação que falhou fica em vermelho.

4. **Clique na ação anterior à falha**
   O centro mostra a tela naquele instante. As abas Before e After mostram antes e depois da ação. A faixa de miniaturas no alto também leva a qualquer momento.

5. **Leia as abas de baixo**
   Errors: o esperado e o recebido. Console: erros do JavaScript da página. Network: as requisições e os códigos (um 500 aponta erro no servidor). Source: a linha do teste.

Também dá para abrir um trace direto pelo terminal:

```
npx playwright show-trace test-results/<pasta-do-teste>/trace.zip
```

O que você deve ver

A mesma janela do Trace Viewer, sem passar pelo relatório.

Para treinar sem esperar uma falha, force o trace num teste que passa. A opção `--trace on` existe no Playwright (confira em `npx playwright test --help`):

```
npm run test:e2e -- -g "PER-02a" --trace on
npx playwright show-report
```

O que você deve ver

O PER-02a passa e, no relatório, ganha a seção Traces. Abra e explore: `goto /alunos`, depois o `expect` do título "Acesso negado".

Vídeo e print

Ficam na pasta `test-results/<nome-do-teste>-chromium/`, ao lado do `trace.zip`: `video.webm` e `test-failed-1.png`. Também aparecem no relatório HTML. A pasta `test-results/` é apagada no começo de

cada execução: copie o que precisa antes de rodar de novo.

Anexar o trace numa issue

1. **Copie o trace.zip para fora do projeto**
   Assim a próxima execução não apaga a evidência.

2. **Confira se não há dado sensível**
   Abra o trace e olhe os prints e a aba Network. O trace guarda as requisições, inclusive o cookie de sessão do usuário de teste.

3. **Abra a issue pelo template Bug**
   Siga o documento 04. Título com o ID: `[PER-02a] Vendedor abre a lista de alunos`. Informe o `sha` de `/api/health` e o comando que você rodou.

4. **Arraste o trace.zip para o campo de texto**
   O GitHub sobe o arquivo e cria o link. Diga na issue que o dev abre com `npx playwright show-trace`.

> **Trace de teste que faz login mostra a senha**
> No setup e no ACE-03, o teste digita a senha real com `fill`, e o trace registra o valor digitado. Não anexe esses traces. Nesse caso, anexe o print e o vídeo e descreva o passo. Os testes que começam logados com `papel` não digitam senha.

7 Anatomia de um teste

Vamos ler o ACE-07 real, de `e2e/login.spec.ts`, pedaço por pedaço. Ele cobre a primeira tentativa do cenário ACE-07 do roteiro: senha errada e e-mail inexistente. Algumas linhas foram quebradas para caber na página.

1. Importar as ferramentas

```
import { test, expect, login, menuPrincipal } from "./fixtures/auth";
import { emailDe, PAPEIS, type PapelE2E } from "./fixtures/usuarios";

const MENSAGEM_LOGIN_RECUSADO = "E-mail ou senha incorretos.";
```

`import` traz funções de outros arquivos. O `test` e o `expect` vêm de `./fixtures/auth`, e não de `@playwright/test`: é essa versão que tem a opção `papel`. A constante guarda o texto esperado num lugar só.

2. Declarar o teste

```
test.describe("Login", () => {
  test(
    "ACE-07 senha errada e e-mail inexistente mostram a mesma mensagem e continuam em /login",
    async ({ page }) => {
```

`test.describe` agrupa testes com um título ("Login"). `test(nome, função)` cria um teste. O nome começa pelo ID. `async ({ page })` recebe a aba do navegador, já aberta e sem sessão, porque este bloco não usa `test.use({ papel })`.

3. Agir na tela

```
      await page.goto("/login");
      await page.locator('input[name="email"]').fill(emailDe("VENDEDOR2"));
      await page.locator('input[name="senha"]').fill("senha-errada-e2e");
      await page.getByRole("button", { name: "Entrar" }).click();
```

`await` quer dizer "espere esta ação terminar antes da próxima". `page.goto("/login")` abre a URL; o começo (`E2E_BASE_URL`) vem da configuração. `locator(...)` encontra um elemento. `fill` digita, `click` clica. O e-mail é o do VEND-B, e a senha é inválida de propósito.

4. Conferir o resultado

```
      await expect(page.getByText(MENSAGEM_LOGIN_RECUSADO)).toBeVisible();
      await expect(page).toHaveURL(/\/login/);
```

`expect(...)` é a asserção. `toBeVisible()` espera até 10 s o texto aparecer. `toHaveURL(/\/login/)` confere que a URL contém `/login` . O que está entre barras é uma expressão regular, um padrão de texto; a `\/` é uma barra escrita de forma "escapada".

5. Repetir com um e-mail que não existe

```
      await page.locator('input[name="email"]').fill("nao-existe-e2e@genius.com");
      // … mesma senha, mesmo clique em Entrar e os mesmos dois expect …

    },
  );
});
```

A mensagem tem de ser a mesma. Se fosse diferente, um invasor descobriria quais e-mails existem no sistema.

Resumo das peças

Peça	Para que serve
`page.goto(url)`	Abre uma página do UAT. Use o caminho (`"/funil"`), nunca a URL inteira.
`page.getByRole(papel, { name })`	Acha pelo papel de acessibilidade e pelo nome visível: botão, link, título, diálogo. A primeira escolha.
`page.getByLabel(texto)`	Acha um campo pelo rótulo escrito ao lado dele.
`page.getByText(texto)`	Acha pelo texto na tela.
`page.locator('input[name="email"]')`	Seletor CSS. Use só quando não há rótulo ou papel, como nos campos do login.
`.fill(texto)` e `.click()`	Digita e clica. Esperam o elemento aparecer, até 15 s.
`expect(...).toBeVisible()`	Confere que está na tela.
`expect(...).toHaveCount(0)`	Confere que não existe.
`expect(page).toHaveURL(...)`	Confere a URL atual.

Seletores têm motivo

O teste usa `getByText`, e não `getByRole("alert")`, porque o Next.js também coloca na página um anunciador de rota com `role="alert"`. Haveria dois "alertas". Quando você escolher um seletor, deixe um comentário se o motivo não for óbvio.

Um teste de permissão com o fixture papel

Em `e2e/permissoes.spec.ts`, cada caso é um objeto numa lista:

```
  {
    id: "PER-02a",
    papel: "VENDEDOR",
    url: "/alunos",
    mecanismo: "na-pagina", // alunos/(lista)/page.tsx: SEC, GP, FIN, PRO
    descricao: "Vendedor não abre a lista de alunos",
  },
```

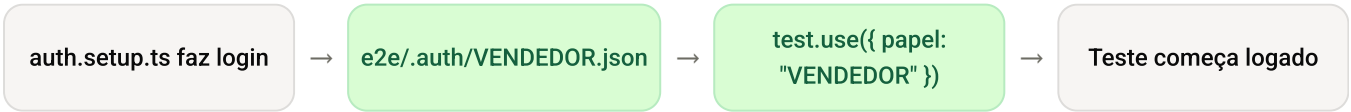
Um laço transforma cada objeto num teste:

```
for (const caso of CASOS_NEGADOS) {
  test.describe(`${caso.papel} em ${caso.url}`, () => {
    test.use({ papel: caso.papel });

    test(`${caso.id} ${caso.descricao}`, async ({ page }) => {
      await page.goto(caso.url);

      await expect(tituloAcessoNegado(page)).toBeVisible();
      if (caso.mecanismo === "redirect") {
        await expect(page).toHaveURL(/\/acesso-negado$/);
        await expect(page.getByRole("link", { name: "Voltar ao início" })).toBeVisible();
      } else {
        await expect(page).toHaveURL(new RegExp(`${escaparRegex(caso.url)}(\\?|$)`));
      }
    });
  });
}
```

A linha que importa é `test.use({ papel: caso.papel })`. Ela troca a sessão do teste pelo arquivo `e2e/.auth/VENDEDOR.json`, salvo pelo setup. O teste começa já logado como João Silva (VEND-A), sem passar pela tela de login. Depois ele digita a URL e confere o título "Acesso negado" e a URL final.



Exercício: a sua primeira contribuição

No roteiro, o PER-02 diz que o VEND-A, ao digitar `/aria`, é negado por redirecionamento. Esse caso não está no spec. Acrescente um objeto à lista `CASOS_NEGADOS`, logo depois do PER-02c: `id: "PER-02e"` (a letra d já é o `test.fixme` de `/financeiro/permuta`), `papel: "VENDEDOR"`, `url: "/aria"`, `mecanismo: "redirect"`, `descricao: "Vendedor não abre o Agente de IA"`. Depois rode `npm run test:e2e -- -g "PER-02e"`. Você deve ver o setup e `1 passed` para o PER-02e.

8 Escrever o seu primeiro teste

Você vai gravar um fluxo com o codegen, limpar o código e criar o arquivo `e2e/crm.spec.ts`. O teste abre o Funil como Vendedor e confere uma parte do CRM-01: a nova oportunidade do vendedor não tem o campo "Responsável".

> **Por que este fluxo**
> Ele não cria dados: abre o diálogo e cancela. É seguro repetir quantas vezes quiser. O CRM-01 completo cria uma oportunidade; isso fica para quando você dominar o básico (seção 10).

1 Garanta que a sessão do Vendedor existe

```
npm run test:e2e -- -g "PER-02a"
```

> **O que você deve ver**
> O setup e `1 passed` no PER-02a. A pasta `e2e/.auth/` passa a ter o arquivo `VENDEDOR.json`.

2 Abra o codegen já logado

O codegen não lê o `.env.e2e`: digite a URL do UAT. A opção `--load-storage` (listada em `npx playwright codegen --help`) carrega a sessão salva, para você não gravar o login com a senha.

```
npm run test:e2e:codegen -- --load-storage e2e/.auth/VENDEDOR.json <url-do-uat>/funil
```

> **O que você deve ver**
> Duas janelas. Um Chromium aberto no Funil, logado como João Silva, com uma barra de ferramentas flutuante no alto. E o Playwright Inspector, que já mostra `import`, `test('test', …)` e a linha `page.goto`.

3 Grave uma conferência do título

Na barra flutuante, clique no ícone de olho (Assert visibility). Depois clique no título "Funil" da página.

> **O que você deve ver**
> Uma linha nova no Inspector: `await expect(page.getByRole('heading', { name: 'Funil' })).toBeVisible();`

4 Abra e feche o diálogo

Clique em **Nova oportunidade**. Use de novo o olho e clique no título do diálogo. Depois clique em **Cancelar**. Não clique em Criar: isso gravaria dados no UAT.

> **O que você deve ver**
> Mais três linhas: o `click` em Nova oportunidade, o `expect` do título e o `click` em Cancelar. O diálogo fecha.

5 Copie o código e feche as janelas

No Inspector, use o botão de copiar. Feche o Chromium e o Inspector. O terminal volta ao prompt.

O código gravado fica parecido com este:

```
import { test, expect } from '@playwright/test';

test('test', async ({ page }) => {
  await page.goto('<url-do-uat>/funil');
  await expect(page.getByRole('heading', { name: 'Funil' })).toBeVisible();
  await page.getByRole('button', { name: 'Nova oportunidade' }).click();
  await expect(page.getByRole('heading', { name: 'Nova oportunidade' })).toBeVisible();
  await page.getByRole('button', { name: 'Cancelar' }).click();
});
```

Ele funciona uma vez na sua máquina, mas ainda não segue o padrão do projeto. Use-o como rascunho.

Limpar o rascunho

Crie o arquivo `e2e/crm.spec.ts` no VS Code e escreva a versão final:

```
// CRM: funil, carteira, contatos e tarefas (módulo 3 do roteiro, docs/qa/03).

import { test, expect } from "./fixtures/auth";

test.describe("Funil como Vendedor", () => {
  test.use({ papel: "VENDEDOR" });

  test("CRM-01 nova oportunidade do vendedor não tem Responsável", async ({ page }) => {
    await page.goto("/funil");
    await expect(page.getByRole("heading", { name: "Funil", exact: true })).toBeVisible();

    await page.getByRole("button", { name: "Nova oportunidade" }).click();
    const dialogo = page.getByRole("dialog", { name: "Nova oportunidade" });
    await expect(dialogo).toBeVisible();
    // CRM-01: "Não existe campo Responsável para o vendedor."

    await expect(dialogo.getByLabel("Responsável")).toHaveCount(0);

    // Fecha sem criar nada: este teste não gera dados no UAT.

    await dialogo.getByRole("button", { name: "Cancelar" }).click();
    await expect(dialogo).toHaveCount(0);
  });
});
```

O que mudou	Por quê
`import` de `./fixtures/auth`	Para ter a opção `papel`.
`test.use({ papel: "VENDEDOR" })`	Começa logado pela sessão do setup. Nada de login gravado.
`page.goto("/funil")`	Só o caminho. A URL do UAT vem do `.env.e2e`, nunca do código.
Nome com o ID `CRM-01`	Liga o resultado ao roteiro, à planilha e à issue.
`exact: true` no título	Evita casar com outro título que contenha "Funil".
`dialogo.getBy…`	Procura só dentro do diálogo. Evita achar outro "Cancelar" na página.
`toHaveCount(0)` do Responsável	O codegen só grava o que existe. A ausência você escreve à mão.
Aspas duplas e comentários	É o estilo dos outros specs.

Rodar o seu teste

```
npm run test:e2e -- e2e/crm.spec.ts
```

O que você deve ver

O setup e uma linha nova, com o ✓:

```
  ✓ 15 [chromium] › crm.spec.ts:7:7 › Funil como Vendedor › CRM-01 nova oportunidade … (3.1s)
```

No fim, `15 passed` (14 do setup e o seu). No VS Code, o teste também aparece na aba Testing.

Prove que o teste sabe falhar

Um teste que nunca falha não testa nada. Troque `"Funil"` por `"Funis"` no `expect` do título, salve e rode o mesmo comando.

O que você deve ver

Um ✗ e o erro abaixo, depois de 10 s de espera:

```
Error: expect(locator).toBeVisible() failed

Locator: getByRole('heading', { name: 'Funis', exact: true })
Expected: visible
Timeout: 10000ms
Error: element(s) not found
```

Desfaça a troca e rode de novo até voltar o ✓. Na planilha de resultados, anote: "CRM-01, parte do campo Responsável, automatizada em `e2e/crm.spec.ts`".

9 Boas práticas neste projeto

Regras curtas que evitam teste frágil, dado perdido no UAT e risco de mensagem real.

ID do roteiro no nome

O nome começa pelo ID (`CRM-04 …`) e segue com uma frase do que se espera. Assim o filtro `-g`, a planilha e a issue falam a mesma língua.

Um spec por módulo

`login.spec.ts`, `permissoes.spec.ts`, `crm.spec.ts` e depois `matricula.spec.ts`, `financeiro.spec.ts`, `alunos.spec.ts`. Não misture módulos num arquivo.

Testes independentes

Os testes rodam em paralelo e em qualquer ordem. Nenhum teste pode depender de outro ter rodado antes, nem de dado criado por outro.

Cada teste cria o que precisa

Se o teste precisa de uma oportunidade, ele mesmo cria, com um nome único, e confere em cima dela.

Seletores: do mais forte ao mais frágil

Ordem	Seletor	Quando usar
1	`getByRole("button", { name: "Entrar" })`	Sempre que der: botões, links, títulos, abas, diálogos.
2	`getByLabel("Responsável")`	Campos de formulário com rótulo.
3	`getByText("E-mail ou senha incorretos.")`	Mensagens e textos soltos.
4	`locator('input[name="email"]')`	Só quando não há papel nem rótulo.
Evite	`locator(".flex > div:nth-child(3) button")`	Classes CSS e posições mudam a cada ajuste de layout. XPath também.

Dados criados pelo teste

O roteiro pede que todo registro comece com `QA`. O README pede que a automação seja identificável. Use o prefixo `QA-E2E-`, que atende os dois, e um carimbo de tempo para o nome não repetir:

```
const nome = `QA-E2E-OPORT-${Date.now()}`;
```

Telefones só fictícios, no formato do roteiro (`+506 8888 01xx`). Como o CRM avisa quando um telefone já existe (CRM-03), peça ao gestor uma faixa reservada para a automação antes de criar oportunidades em massa.

Faça

- Espere com `expect(...)`. Ele já espera até 10 s o elemento aparecer.
- Escreva o bug conhecido com o comportamento esperado e marque com `test.fixme`, com um comentário citando o X ou o R.
- Comentários e nomes em português.
- Rode o seu teste novo pelo menos três vezes seguidas antes do PR. Se falhar uma vez sem motivo, ele está instável.
- Quando um texto da tela mudar, confirme com o roteiro ou com o dev se a mudança é intencional antes de mudar o teste.

Não faça

- `page.waitForTimeout(...)` ("espere 3 segundos"). Deixa o teste lento e ainda falha quando o UAT demora mais.
- `test.only` esquecido no commit. Ele faz só aquele teste rodar.
- Alterar configuração global do UAT (Operação, Países, Catálogo, Preços, Usuários) num teste automático sem combinar. Isso muda o ciclo manual dos colegas.
- Mexer em `vitest.config.ts` ou nos testes de `src/`. São do time de dev.

Nunca, em hipótese alguma

Nunca rode contra produção. `E2E_BASE_URL` é sempre o UAT. **Nunca escaneie o QR do WhatsApp**, nem à mão nem num teste, e nunca automatize o botão "Conectar" de um número (CFG-30). Algumas ações da Inbox chamam o motor do WhatsApp direto, sem a trava `WHATSAPP_LIVE`. Com um celular pareado, elas mandariam mensagem de verdade (bug X15). Nunca use e-mail, telefone ou documento de pessoa real.

10 O que automatizar primeiro

Comece pelos P1 que não criam dados. Depois, os que criam dados com `QA-E2E-`. Deixe de fora o que mexe na configuração compartilhada e o que esbarra num bloqueador.

Onda 1: sem criar dados

Cenário	O que automatizar	Situação	Observação
ACE-03, ACE-06, ACE-07, ACE-13	Menu por papel; rotas sem sessão; senha errada; Vendedor em `/alunos`	Já coberto	ACE-03, ACE-06 e ACE-07 em `login.spec.ts`; ACE-13 pelo PER-02a. O ACE-07 cobre só a 1ª tentativa.
ACE-01	`/api/health` com `ok: true` e `/api/ready` com `ready: true`	Dá hoje	Sem login. Use o request do Playwright, sem navegador.
PER-02, PER-09, PER-11, PER-13	URLs que devem ser negadas	Em parte	Já há PER-02a–c, PER-09a–b, PER-11a–b e PER-13a–b. Acrescente as URLs que faltam em `CASOS_NEGADOS`, com a próxima letra livre. URLs com `<id>` da massa ficam para depois.
PER-01, PER-04, PER-06, PER-08, PER-10, PER-12	URLs que devem abrir para o papel	Dá hoje	Confira o título da página e a ausência de "Acesso negado". Pede um helper novo; peça revisão ao dev.
PER-05, PER-07, PER-04	Menu Empresas, tela de erro, Configuração › CRM	fixme	PER-05a e PER-07d (X2), PER-02d e PER-11c (X3), PER-04a (R13) já estão no spec. PER-07a–c já testam as URLs negadas da Secretaria.
CFG-24, passo 1	ADM tenta desativar a si mesmo e vê "Você não pode desativar a si mesmo."	Dá hoje	Não confirme nada além da mensagem.

Onda 2: cria dados com QA-E2E-

Cenário	O que automatizar	Situação	Observação
CRM-01	Vendedor cria oportunidade; cartão em Novo, código `L-` e 6 dígitos, dono João Silva	Dá hoje	Depende do CFG-20 feito no UAT.
CRM-04	GC sem responsável recebe erro; com responsável cria	Dá hoje	Papel `GERENTE_COMERCIAL`.
CRM-05	VEND-A não vê cartões do VEND-B, nem pela URL	Dá hoje	Dois papéis no mesmo teste. Peça ajuda ao dev na primeira vez.
CRM-10	"Preparar contratação" abre a tela e a oportunidade fica em Proposta	Dá hoje	O teste cria e avança a própria oportunidade.

Cenário	O que automatizar	Situação	Observação
CRM-14, CRM-22	GC troca o responsável; cobertura de carteira	Dá hoje	Revogue a cobertura no fim do teste.
CFG-25, CFG-26, CFG-28	Modelo contratual; números de WhatsApp fictícios	Com cuidado	Versões publicadas ficam para sempre. No CFG-28, nunca clique em Conectar.
CFG-22	E-mail com maiúscula	fixme	Bug X1. Desative os usuários criados no fim.

Ainda não

Cenário	Motivo
CFG-02, CFG-03, CFG-05, CFG-14, CFG-18, CFG-19, CFG-20	Preparação do ambiente. Muda configuração usada por todo o ciclo manual. Fica manual, feita uma vez.
ACE-10, ACE-11	Tira papel ou desativa um usuário compartilhado no meio de testes paralelos. O ACE-10 usa o FIN+SEC, que não tem variável.
CFG-30	Mostra o QR do WhatsApp. Nunca automatizar (X15).
CRM-08	Bloqueador B6: a experimental não chega ao professor.

O que não dá para automatizar hoje

Os bloqueadores da auditoria consolidada (seção 2 dela) travam o fluxo pela interface. Sem o fluxo, não há o que o teste clicar.

#	O que trava	Por que o teste não passa	Quando revisitar
B2	Assinatura de contrato	Não existe driver de assinatura, nem simulado. A matrícula do fluxo novo não chega a ATIVA.	Quando o dev entregar o driver simulado ou a fixture.
B3	Pagamento	O pagamento simulado está desligado no código; o webhook responde 404; nenhuma tela gera o link `/pagar`.	Depois da decisão R2 e da correção da flag.
B4	Portal do aluno	O token de ativação só nasce com e-mail enviado de verdade. Ativação, login e recuperação ficam intestáveis.	Quando o PO escolher a caixa de teste ou a ferramenta de token.
B5	Gravação de aula	Exige Google Drive. Sem ele, "Erro inesperado". Sem aula Ministrada, não há reposição nem conclusão.	Quando houver Drive de teste.
B6, B8	Experimental e mensagem recebida	Etapas do funil inalcançáveis; não há como simular mensagem recebida no WhatsApp sem o dev.	Depois da correção do B6 e da receita do B8.

11 Quando quebrar

Os erros mais comuns, a mensagem que aparece e o que fazer. Antes de abrir bug, descarte os problemas de configuração desta lista.

O que aparece	Causa e o que fazer
`[e2e] E2E_BASE_URL não definida`	Nada roda. O `.env.e2e` não existe, está em outra pasta ou a linha está vazia. Ele fica na raiz, ao lado do `playwright.config.ts`. Refaça a seção 4.

O que aparece	Causa e o que fazer
`[e2e] Senha do papel X não definida` ou `[e2e] Sessão de X não encontrada`	Preencha `E2E_PASSWORD_X` no `.env.e2e`. No modo UI, rode antes uma vez `npm run test:e2e` pelo terminal, para o setup criar as sessões.
`page.goto: Protocol error (Page.navigate): Cannot navigate to invalid URL`	A `E2E_BASE_URL` está sem `https://` ou com erro de digitação. Copie de novo a URL que o gestor passou.
`✗ [setup] › … › autenticar VENDEDOR2`, `page.waitForURL: Timeout 30000ms exceeded` e, no fim, `33 did not run`	O login daquele papel não chegou à Home, e o projeto `chromium` inteiro parou. Causas, em ordem: senha errada; e-mail bloqueado por 5 falhas (linha abaixo); usuário desativado no UAT. Abra o trace para ver a tela de login, mas não o anexe: ele mostra a senha.
"E-mail ou senha incorretos." com a senha certa	Bloqueio por 5 falhas em 15 minutos no mesmo e-mail. O ACE-07 gasta uma tentativa do VEND-B por execução da suíte. O bloqueio fica na memória do servidor e some 15 minutos depois da primeira falha, ou quando o servidor reinicia. Espere e passe a filtrar (`-g` ou arquivo). Se não sair, fale com o gestor.
`Test timeout of 60000ms exceeded.`	UAT lento ou fora do ar. Abra `<url-do-uat>/api/health`. Se não responder, avise o gestor e pare. Se responder, rode só o que falhou: `npm run test:e2e -- --last-failed`. Se o mesmo teste sempre estoura, abra o trace.
`expect(locator).toBeVisible() failed` … `element(s) not found`, ou `locator.click: Timeout 15000ms exceeded`	O texto da tela mudou, o papel não vê o elemento (pode ser bug de permissão) ou a página deu erro antes (abas Console e Network do trace). Só ajuste o teste se a mudança for intencional. Se for bug, abra issue.
`strict mode violation: getByRole('button', { name: 'Cancelar' }) resolved to 2 elements`	O seletor achou mais de um elemento. Use `exact: true` ou procure dentro de uma região: `page.getByRole("dialog", …).getByRole("button", …)`.
`browserType.launch: Executable doesn't exist at …ms-playwright…`	Falta o navegador. Rode `npx playwright install chromium`. Acontece depois de atualizar o Playwright do projeto.
ACE-03 falha num papel só	Menu com item a mais, a menos ou fora de ordem. Pode ser bug, ou o `MENU_ESPERADO` de `login.spec.ts` ficou para trás em relação a `src/lib/nav.ts`. Compare com a tabela "Menu esperado por papel puro" do roteiro e pergunte ao dev antes de mudar o teste.
Passa às vezes, falha às vezes	Teste instável (flaky). Não resolva com `waitForTimeout`. Abra o trace e procure o que muda: mensagem que some rápido, dado de outro teste, lista ainda carregando. Espere o estado certo com `expect` e avise no PR.
`npx.ps1 não pode ser carregado` (PowerShell)	Use `npx.cmd` e `npm.cmd`, ou o terminal Git Bash (seção 3).

12 Fluxo de entrega

O teste só vale para o time quando chega à `main` por um pull request. Você commita apenas arquivos da pasta `e2e/`.

1 Atualize a main e crie a sua branch

```
git checkout main
git pull
git checkout -b qa/e2e-crm-01
```

O que você deve ver

`Switched to a new branch 'qa/e2e-crm-01'`. Use o prefixo `qa/` e o ID ou o módulo no nome.

2 Com o teste estável, rode o lint

Estável é pelo menos três execuções seguidas com ✓, filtrando pelo seu arquivo.

```
npx eslint e2e
```

O que você deve ver

Nenhuma saída: está tudo certo. Se aparecer uma lista de avisos ou erros, corrija antes do commit. Erros de TypeScript aparecem sublinhados em vermelho no VS Code e na aba Problems.

3 Confira o que vai no commit

```
git status
```

O que você deve ver

Só arquivos dentro de `e2e/`, como `e2e/crm.spec.ts`. Nunca `.env.e2e`, `e2e/.auth/`, `test-results/` ou `playwright-report/`. Se aparecer arquivo fora de `e2e/`, não inclua.

4 Adicione pelo caminho e faça o commit

Use o caminho do arquivo, nunca `git add .`.

```
git add e2e/crm.spec.ts
git commit -m "e2e: CRM-01 nova oportunidade do vendedor não tem Responsável"
```

O que você deve ver

Uma linha com a branch, o código curto do commit e a mensagem, e `1 file changed`.

5 Envie a branch e abra o PR

```
git push -u origin qa/e2e-crm-01
```

O que você deve ver

O Git mostra um link para criar o pull request. Abra o link, confira que a base é `main` e preencha a descrição.

O que escrever na descrição do PR

Os IDs do roteiro cobertos (total ou parcial); o comando, o resultado (ex.: `15 passed`), a data e o `sha` de `/api/health`; os `test.fixme` com o X ou R de cada um; e os dados que o teste cria no UAT. Nunca senha, URL do UAT ou trace com dado sensível.

A CI não roda os testes E2E

A CI do repositório (`.github/workflows/ci.yml`) roda quatro coisas em cada PR: typecheck, lint, testes unitários (em dois fusos) e build. Ela não tem o `.env.e2e` nem acesso ao UAT, e não executa o Playwright. CI verde não quer dizer que o seu teste passa: quem revisa confia no resultado que você colou no PR. Mas a CI confere os seus arquivos: o TypeScript inclui a pasta `e2e/` , e o lint tem um teto de 40 avisos no projeto todo, que um aviso novo seu pode estourar.

13 Glossário e checklists

Glossário

Asserção	A conferência de um teste, escrita com `expect(...)` . Se não se confirma no tempo, o teste falha.
baseURL	O começo de todas as URLs, vindo de `E2E_BASE_URL` . Por isso o teste escreve só `"/funil"` .
Codegen	Gravador do Playwright. Abre o navegador e escreve o código enquanto você clica.
Fixture	Algo que o Playwright prepara para o teste antes de ele começar: a `page` , a sessão do `papel` .
Flaky	Teste instável: às vezes passa, às vezes falha, sem mudança no sistema.
Locator	A "mira" que encontra um elemento na tela, como `getByRole(...)` .
Papel	O usuário de teste que o teste usa: `VENDEDOR` , `PROFESSOR` . Configurado com `test.use({ papel })` .
Rate limit	Limite de tentativas. No login: 5 falhas em 15 minutos por e-mail.
Sessão (storageState)	Os cookies de um login, salvos em `e2e/.auth/<PAPEL>.json` .
test.fixme	Teste listado e não executado, para bug conhecido ou decisão pendente.
Trace	Gravação completa de um teste (`trace.zip`), aberta no Trace Viewer.
UAT	Ambiente de homologação, onde o QA testa. Nunca é produção.

Checklist da primeira semana

- [ ] `node -v` mostra `v22` e `npx playwright --version` mostra `1.64.1` .
- [ ] O `.env.e2e` está preenchido e `git check-ignore -v .env.e2e` confirma a proteção.
- [ ] `npx playwright test --list` mostra `Total: 50 tests in 3 files` .
- [ ] `npm run test:e2e` termina com 45 passaram e 5 pulados.
- [ ] Rodei com `-g` e `--headed` , e abri um trace com `--trace on` .
- [ ] Fiz o exercício do PER-02e e o `e2e/crm.spec.ts` da seção 8.
- [ ] Fiz o teste falhar de propósito e voltei ao ✓.
- [ ] Sei a quem pedir ajuda: gestor ou dev de plantão, com o comando, o erro completo e o `sha` do UAT, sem senha nem trace de login.

Checklist antes de abrir o PR

- [ ] O nome de cada teste começa pelo ID do roteiro, e nenhum teste depende de outro.
- [ ] Dados criados começam com `QA-E2E-` e usam só telefones e e-mails fictícios.

- [ ] Nenhuma senha, URL do UAT, `test.only`, `waitForTimeout`, QR ou "Conectar" de número no código.
- [ ] Três execuções seguidas com ✓, `npx eslint e2e` sem saída e nada sublinhado em vermelho.
- [ ] `git status` mostra só arquivos de `e2e/`.
- [ ] A descrição do PR tem IDs, comando, resultado, data e `sha` do UAT.